

L4 Compiler: Design, Implementation, and Optimization

Viraj Shah and Samriddhi Bhardwaj

June 1, 2026

1 Executive Summary

Configuration	Total score	Avg. safe multiplier	Avg. unsafe multiplier
-00	0.318	1.167	0.224
-01	0.673	1.470	0.584

Table 1: Overall compiler performance summary.

2 Compiler Optimization Structure

Frontend

parse header/source -> elaborate -> typecheck(with string interner)

Middle end — SSA IR tree

```
elaborated AST -> ast_to_ir (--unsafe skips dynamic checks*)
-> [ -01 only: ir_opt (**) ]
-> ir_to_asm (abstract asm)
-> asm_to_llvm (LLVM-like SSA IR)
-> [ -01 only: llvm_optim (***), using llvm_analysis ]
-> llvm_to_3addr (3-address x86 SSA)
* --unsafe: in ast_to_ir_tree.rs omit null/bounds/shift/array-length
  checks etc. (+ LLVM basic opts allow extra div/rem rewrites).
** ir_opt (ir_opt.rs, fixed-point per function until stable):
  mov constant/copy propagation -> constant fold / algebraic strength
  on PureExpr -> constant Cjumps -> Goto ->
  eliminate redundant Goto(Marker(next)) -> dead-code eliminate
  dead Mov/Sext/Phi definitions
*** llvm_optim (/llvm_optim/mod.rs --- order abbreviated):
  multiple rounds of: basic.optimize (llvm_optim/basic.rs)
    [propagate, strength, branch fold, DCE (+ label coalesce)]
-> function inline -> scalar replacement (stack alloc promotion)
-> tail-call optimization -> LICM (***)-> bounded PRE iterations
    (llvm_optim/pre.rs) -> induction-variable strength reduction
-> LICM again + final basic cleanups.
llvm_analysis/: dominators on LLVM CFG (dense bitvectors); natural-loop
detection; block layout parsing; ptr-origin facts for LICM safety.
```

Backend — 3-address x86 SSA

insert ABI conventions -> build CFG -> split critical edges

```

-> CMov if-conversion -> liveness -> graph-coloring register alloc
-> SSA deconstruction (parallel copies) -> post-regalloc ABI fixups
-> x86.rs: lower to 2-address AT&T x86 (prologue/frame/emission)

```

Figure 1: End-to-end pipeline. `ir_opt` and `llvm_optim` run only when `-O1` or default optimizer level 1. `-O0` disables both; codegen passes after `llvm_to_3addr` still run.

3 Implemented Optimizations

3.1 Arithmetic Strength Reduction

Source file: `llvm_optim/basic.rs`

Arithmetic strength reduction replaces expensive arithmetic with cheaper equivalent instruction sequences. Our pass handles signed constant division using the Granlund–Montgomery multiplier/shift method, reconstructs constant remainders as $r = n - qd$, replaces multiplication by powers of two with shifts, and expands selected small constant multiplications into shifts/adds/subs. We specialize to signed quotient-rounded-toward-zero division because C0 integers are signed, following the standard invariant-division techniques described by Granlund and Montgomery and Warren [1, 2]. We also preserve trapping behavior by not constant-folding or dead-code-eliminating `SDiv/SRem` operations.

Table 2 compares full `-O1` against an ablation with strength reduction disabled. Positive Δ cycles means the pass improved runtime; positive instruction deltas mean the final assembly contains fewer expensive arithmetic instructions.

Benchmark	Cycles w/o SR	Cycles w/ SR	Δ cycles	Δ div/rem	Δ imul	Pattern
<code>julia.14</code>	2,435,100,125	1,628,367,583	806,732,542	18	12	mixed
<code>arrays_and_loops.14</code>	937,212,046	919,308,880	17,903,166	3	-2	div/rem
<code>daisy.14</code>	191,865,666	182,771,052	9,094,614	16	14	mixed
<code>yyb.14</code>	189,105,390	195,521,161	-6,415,771	67	-14	div/rem
<code>fannkuch.14</code>	2,006,807,979	2,063,260,531	-56,452,552	0	3	pow2 mul

Table 2: Effect of arithmetic strength reduction on representative benchmarks.

The largest gain is on `julia.14`, which contains hot numeric code with constant divisions by values (2, 4, 255), plus Collatz-style operations like `n % 2`, `n / 2`, and `3*n+1`. Since these execute repeatedly inside rendering and recursive Collatz loops, replacing them has a large dynamic effect. `arrays_and_loops.14` and `daisy.14` also improve because they contain repeated constant arithmetic in loop-heavy code.

3.2 Function Inlining

Source file: `llvm_optim/inline.rs`

Function inlining replaces selected call sites with the callee body. The main goal is to remove call overhead and expose the callee to later scalar optimizations such as propagation, CSE, branch folding, and DCE. Our inliner is deliberately conservative: non-recursive callees are inlined only when their body has at most 22 non-structural instructions, mutually recursive cycles are skipped, and directly self-recursive callees are allowed a larger limit of 52 instructions but only with a recursive inline budget of one. This lets us

capture small helper functions and one-step recursive simplifications without allowing unbounded code growth.

Table 3 compares full `-O1` against an ablation with inlining disabled. Positive Δ cycles means inlining improved runtime. IR Δ calls counts call sites removed in the IR, while asm Δ call counts final assembly call instructions removed.

Benchmark	Cycles w/o inline	Cycles w/ inline	Δ cycles	IR Δ calls	asm Δ calls	Pattern
julia.14	3,457,622,458	3,369,989,083	87,633,375	5	7	self-recursive
ncik.14	917,328,697	858,972,687	58,356,010	2	7	self-recursive
yyb.14	433,866,776	380,548,020	53,318,756	3	35	helper in loop
daisy.14	186,995,479	186,478,364	517,115	13	37	small helpers
jack.14	52,247,338	52,124,073	123,265	2	5	small helpers
georgy.14	124,583,833	132,844,333	-8,260,500	-1	40	dense calls

Table 3: Effect of function inlining on representative benchmarks.

The largest improvements come from benchmarks where small functions are called repeatedly in hot code. In `julia.14`, inlining helps inside numeric and recursive Collatz-style kernels, where eliminating even a few call sites matters because the functions are invoked many times. `yyb.14` also benefits substantially: its hot loops repeatedly call small arithmetic and predicate helpers, so inlining removes many final assembly calls and exposes more local simplification opportunities.

Inlining is especially useful as an enabling optimization because it exposes callee bodies to later propagation and dead-code elimination [3]. Its direct effect is removing call overhead, but its larger value is exposing specialized code to the repeated scalar cleanup passes that follow it in the optimization pipeline.

3.3 Tail-Call Optimization

Source files: `llvm_optim/tco.rs`

We implement direct self-tail-recursion elimination in `llvm_optim/tco.rs`, invoked from `llvm_optim/mod.rs`. The pass follows the standard tail-recursion-elimination idea of replacing self-recursive tail calls with jumps back to a loop header [5]. The pass finds adjacent patterns `Call self; Ret` where the returned value is exactly the call’s result temp (non-void), or void `Call` followed by void `Ret`. It rewires these into an explicit loop header with ϕ -nodes over parameters, replacing parameter uses in the body with those ϕ -temps—so recursion depth no longer grows the stack.

Not transformed by this pass: classic factorial after mult: `return n * fact(n - 1);` since the work after the recursive call breaks the tail pattern.

Benchmarks: Heavy tail-recursion appears in `tests/bench/bench_2/pierre.14` (`pow`, `fermat`)

Build	Avg. best cycles		Wall time	
-O1, before TCO (<code>llvm_tco</code> disabled)	22,752,955		0,171 s	
-O1, after TCO (default)	18,536,307	(-18.5 %)	0,112 s	(-34.5 %)
GCC -O1	23,856,894		0,173 s	

Table 4: `pierre` benchmark: TSC figure from `bench.c` (stdout); wall time from `time` on the full process (single run). Lower is better. Unsafe mode. Percentages on the highlighted row are change vs. `-O1` before TCO (negative = reduction).

3.4 Loop and Memory Optimizations

Source files: `llvm_optim/licm_pass.rs`, `pre.rs`, `iv_strength.rs`, `scalar_replacement.rs`

LICM Hoists *loop-invariant* work (loads, arithmetic, etc.) into a preheader when dominance and memory/pointer-origin checks allow it. Strong when inner loops recompute values that depend only on outer indices or globals.

PRE Conservative partial redundancy elimination for duplicate `BinOp`, `PtrAdd`, and `Sext` when one copy dominates another or a single-predecessor merge permits insertion; diamond merges that would need ϕ nodes are skipped.

IV strength Replaces `mul` of an induction variable by a small constant (in the loop header) with an auxiliary ϕ updated by `add` on the latch. Only matches tight affine latch patterns.

Scalar replacement Promotes small stack-like `alloc` regions to SSA scalars only in very restricted loops (single outside predecessor, single latch, no `CondBr` in the loop body except the header, safe pointer uses). Often a poor match on large, branchy benchmarks.

Pipeline note. Passes compose: scalar replacement and subsequent `basic` cleanup can simplify bodies for LICM; PRE removes redundancy before IV strength; a second LICM runs after IV strength.

Loop and memory optimizations interact strongly with program shape and not just program size. LICM and PRE are the broadest wins on the `bench_1` suite: nested numeric kernels (`arrays_and_loops`, `daisy`), and pointer-heavy structures (`danny`) exhibit repeated address arithmetic or recomputed pure values inside loops where hoisting and partial redundancy removal apply. Affine IV strength reduction is more selective: it helps when lowering produces a recognizable `mul` of an induction variable by a constant in a simple latch/header pattern which is common in tight numeric inner loops but less central in pointer-heavy tree code. Scalar replacement is conservative in our implementation (single-entry, single-latch, no internal `CondBr` in the loop body), so large benchmarks rarely match it; when it does fire, it mainly prepares cleaner loops for later `basic` and LICM rounds rather than dominating wall-clock by itself.

File	Cycles (off)	Cycles (on)	Δ_{cyc}
<code>fannkuch.14</code>	10,553,529,830	10,264,580,003	288,949,827
<code>arrays_and_loops.14</code>	3,569,027,369	3,495,516,415	73,510,954
<code>daisy.14</code>	1,253,738,365	1,238,817,647	14,920,718
<code>jen.14</code>	4,707,557,445	4,687,318,525	20,238,920
<code>danny.14</code>	553,485,279	551,411,264	2,074,015
<code>frank.14</code>	199,033,653	198,738,192	295,461
<code>georgy.14</code>	605,752,146	605,772,742	-20,596
<code>jack.14</code>	368,698,528	377,205,994	-8,507,466
<code>albert.14</code>	1,134,798,053	1,169,915,587	-35,117,534
<code>janos.14</code>	835,182,050	982,223,316	-147,041,266

Table 5: `bench_1` loop/memory LLVM ablation under `-unsafe`. Row color shows relative cycle improvement. Green means faster with the pass enabled; red means slower; pale means nearly unchanged.

Those programs therefore behave like the “easy” cases the bundle targets. At the other extreme, Table ?? shows clear losses on `janos` and `albert`: the former is dominated by hashtable pointer chasing rather than a single stable array induction, and the latter is a tight, spill-sensitive kernel where small IR changes can outweigh loop wins. Smaller positive or near-zero rows in the same table are consistent with workloads where only part of the runtime is loop-shaped or where wins and layout effects nearly cancel.

3.5 Backend If-Conversion

Source file: `cmov_if_convert.rs`

Our backend if-conversion pass turns tiny branch diamonds into straight-line `Select` instructions, which later lower to x86 `cmov`. The pass only converts diamonds where both arms are empty or contain one `Mov`, and the merge block begins with `phis` from those two arms. This keeps the optimization conservative: `cmov` can remove branch overhead, but it can also be worse when the original branch was predictable. More generally, if-conversion is a standard control-flow transformation that trades branches for predicated or conditional operations [3].

Benchmark	Δ cycles	% Δ cycles	Diamonds	Δ jumps	Δ <code>cmov</code>
<code>loooops.14</code>	76,653,057	3.60%	3	9	3
<code>yyb.14</code>	57,644,855	13.26%	13	39	13
<code>frank.14</code>	-18,348,448	-28.25%	19	57	19

Table 6: Effect of backend if-conversion on representative benchmarks.

Overall, if-conversion is useful but context-sensitive. Our implementation is therefore intentionally narrow: it only converts tiny branch-plus-phi diamonds and leaves larger control-flow regions unchanged.

3.6 Register Allocation

Source files: `regalloc.rs`, `abi_conventions.rs`

Our backend uses graph-coloring register allocation with move coalescing. We added a small pre-spill heuristic for high-pressure functions: after coalescing, the allocator approximates a large clique using maximum-cardinality search and, if the clique needs more registers than available, pre-spills a few low-cost temporaries before greedy coloring. Spill cost is weighted by loop nesting depth, so values used in deeper loops are less likely to be selected for spilling. This pre-spill strategy is motivated by graph-coloring allocation techniques that reason about large cliques before coloring [4].

Table 7: Effect of the pre-spill register-allocation heuristic.

Benchmark	Δ cycles	% Δ cycles	Spills w/o	Spills w/	Pre-spills	Stack Δ
<code>fannkuch.14</code>	384,058,135	15.54%	10	5	5	0
<code>julia.14</code>	11,453,609	0.66%	45	41	28	0
<code>ncik.14</code>	-49,863,020	-13.24%	15	12	8	0

The pre-spill heuristic helped `fannkuch.14`, cutting spills from 10 to 5 and improving cycles by 15.5%. However, the effect was not consistently positive: `ncik.14` also reduced spills but regressed, showing that fewer spills is not always better if the remaining spills or extra moves land on hotter paths. Our register-allocation changes were useful engineering improvements, but they only gave a modest overall benchmark boost, roughly a few percent, so we kept the more experimental call-clobber relaxation disabled and would need better heuristics before enabling it by default.

3.7 Final x86 Peephole Optimizations

Source file: `x86.rs`

The final peephole pass runs after lowering to 2-address x86-64. Unlike the middle-end passes, it does not change the program structure; it only removes local instruction inefficiencies introduced by lowering. The implemented rewrites include folding `mov $0, %r; cmp %r, x` into `cmp $0, x`, replacing `cmp $0, r` with `test r, r`, removing unconditional jumps to the immediately following label, and fusing `leaq` followed by a load/store through the temporary into a single memory operation with displacement. These are small local rewrites, but they occur frequently after CFG lowering and SSA deconstruction.

Table 8 compares full -O1 against an ablation with final x86 peepholes disabled. Positive Δ cycles means the peephole pass improved runtime. Δ static counts the reduction in final static instruction count, while the individual peephole columns show which patterns were responsible.

Table 8: Effect of final x86 peephole optimizations on representative benchmarks.

Benchmark	Δ cycles	% Δ cycles	Δ static	Peepholes	<code>cmp</code> → <code>test</code>	Dead jumps
<code>loooops.14</code>	524,435,098	20.53%	77	97	20	77
<code>fannkuch.14</code>	256,754,292	11.22%	40	45	5	40
<code>julia.14</code>	161,902,385	9.00%	99	105	6	99
<code>yyb.14</code>	115,183,464	36.80%	218	291	73	215
<code>georgy.14</code>	52,369,521	30.63%	236	270	34	230
<code>daisy.14</code>	24,260,558	11.62%	216	244	28	211

The dominant win comes from dead-jump elimination. Our earlier lowering stages often produce explicit jumps even when the target label is immediately next in the final instruction stream. Removing these jumps improves fall-through behavior and reduces the number of executed control-flow instructions. This is especially helpful in loop-heavy benchmarks such as `loooops.14`, where many simple loop and branch blocks are executed repeatedly, and in control-flow-heavy benchmarks such as `yyb.14` and `georgy.14`.

The `cmp` to `test` rewrite is smaller but consistently useful: zero comparisons are common after lowering boolean and null/integer checks, and `test r,r` is the standard shorter idiom for checking whether a register is zero. The `leaq` fusion rule appears less often in these measurements, but it removes temporary address computations when the temporary is used only for an immediate load or store.

4 Effect of `-unsafe`

Benchmark	Safe cyc	Unsafe cyc	Δ cyc	$\Delta\%$
<code>fannkuch.14</code>	51 586 764 340	20 552 788 362	31 033 975 978	+60.16%
<code>daisy.14</code>	4 039 861 666	2 068 850 505	1 971 011 161	+48.79%
<code>mat.14</code>	2 528 046 608	668 112 201	1 859 934 407	+73.57%
<code>janos.14</code>	3 643 229 045	2 244 759 850	1 398 469 195	+38.39%
<code>albert.14</code>	2 355 445 141	1 153 882 050	1 201 563 091	+51.01%
<code>yyb.14</code>	2 242 557 451	1 105 886 225	1 136 671 226	+50.69%
<code>loooops.14</code>	11 533 752 527	10 463 775 670	1 069 976 857	+9.28%
<code>danny.14</code>	1 911 385 150	947 986 728	963 398 422	+50.40%
<code>jack.14</code>	1 225 483 871	739 868 793	485 615 078	+39.63%
<code>ronald.14</code>	887 417 101	564 872 948	322 544 153	+36.35%
<code>georgy.14</code>	1 515 691 471	1 234 951 587	280 739 884	+18.52%
<code>julia.14</code>	14 172 257 563	13 925 705 831	246 551 732	+1.74%
<code>ncik.14</code>	2 626 898 625	2 393 974 776	232 923 849	+8.87%
<code>frank.14</code>	558 344 288	398 574 138	159 770 150	+28.61%
<code>mist.14</code>	5 632 874 956	5 618 979 585	13 895 371	+0.25%
<code>pierre.14</code>	20 303 428	12 862 159	7 441 269	+36.65%
<code>monica.14</code>	755 469 138	759 812 552	-4 343 414	-0.57%
<code>jen.14</code>	9 138 708 313	9 321 584 223	-182 875 910	-2.00%
<code>arrays_and_loops.14</code>	7 791 684 782	8 625 231 222	-833 546 440	-10.70%

Table 9: Safe vs. `-unsafe` at `-O1` (KBEST cycles, `bench.c`). Suite `tests/bench`.

Across `tests/bench`, `-unsafe` usually lowers KBEST cycle counts; the starkest gaps appear in array- and permutation-heavy programs such as `fannkuch.14` and `mat.14`, where safe builds pay a large recurring cost that unsafe can fold away. `fannkuch.14` allocates three `int[]` buffers of length `n` and spends essentially all of its time in nested `while/for` loops that repeatedly read and write `perm[...]`, `perm1[...]`, and `count[...]` via XCH (lines 45–94), so a large safe-vs-unsafe cycle gap is exactly what you expect when every subscript in that hot permutation walk can carry a dynamic bound check in the safe configuration. `mat.14` is dominated by `matrix_mult`’s triple nested loop over 500×500 matrices (lines 51–60), i.e. millions of `M[r][c]` style accesses plus recursive `roll_col/roll_row` that still index the same matrices, which again ties the benchmark’s cost directly to repeated bounds-checked indexing in safe mode. `mist.14` fixes `size()` at 64 and only ever fills `int[]` buffers of that length inside `workUnit` (lines 37–48, 55–72), so after constant propagation many indices are tiny-range and the measured $\Delta\%$ is essentially zero, consistent with little remaining check overhead in either build. `julia.14` does scan two $\text{DIM} \times \text{DIM}$ images by `offset = x + y * DIM` (e.g. lines 109–117, 124–140), but the same routines also mix in substantial scalar work (`collatz`, recursive variants, and the inner `cmplx *` loop in `julia`), so array checks are only part of the story and a small positive $\Delta\%$ is plausible without appealing to unspecified backend effects.

5 Optimization Interactions

Many passes are not independent: inlining enables scalar cleanup, strength reduction depends on DCE and register allocation, loop optimizations expose and consume each other’s results, and if-conversion trades fewer branches for potentially higher register pressure. Below we explore some interesting optimization interactions.

5.1 Early SSA IR-Tree Optimizations

Source files: `ir_opt.rs`

This pass runs before LLVM-like IR generation at `-O1`. It performs fixed-point cleanup over the SSA IR tree, including mov-chain propagation, constant propagation, constant folding, algebraic simplification, conditional-branch folding, dead fall-through jump elimination, and local dead-code elimination. The SSA-like representation makes these propagation and dead-definition cleanups easier because each temporary has a single definition [6].

The purpose of this stage is to simplify the program before later lowering stages. For example, folding a comparison to a constant can turn a conditional jump into an unconditional jump, and removing dead definitions reduces the number of temporaries seen by later stages.

Table 10: Eight benchmarks where disabling `ir_opt` increased KBEST cycles (measurement from `compiler/log/ir_opt_all.json`, `bench.c_ir_opt_ablation`, `-O1 -unsafe, -disable-opts=ir_opt` vs default).

File	cyc (no <code>ir_opt</code>)	cyc (with)	Δ cyc
<code>mist.14</code>	5 595 367 911	5 496 208 161	99 159 750
<code>arrays_and_loops.14</code>	3 574 294 373	3 515 051 254	59 243 119
<code>mat.14</code>	662 247 800	651 024 636	11 223 164
<code>monica.14</code>	755 498 959	747 155 934	8 343 025
<code>yyb.14</code>	1 124 358 972	1 116 169 649	8 189 323
<code>jack.14</code>	371 329 079	367 170 044	4 159 035
<code>pierre.14</code>	14 732 638	12 627 969	2 104 669
<code>frank.14</code>	198 342 419	198 170 480	171 939

Later stages repeat similar cleanup on LLVM IR (`lab5/src/llvm_optim/basic.rs`: propagate, CSE, DCE, branch fold, etc.), invoked multiple times from `lab5/src/llvm_optim/mod.rs` around inlining, LICM, PRE, and other passes. They are not redundant with `ir_opt`: they run on flat `Instrs` after lowering, while `ir_opt` runs on the IR tree (`lab5/src/ir_opt.rs`) and shrinks pure work and obvious jumps before that translation. Doing that shrinkage first means less abstract assembly and a smaller LLVM program, so downstream fixpoints and size-sensitive heuristics (e.g. LICM rounds tied to instruction count in `lab5/src/llvm_optim/licm_pass.rs`) search a smaller space. In short, `ir_opt` does not replace the LLVM bundle; it removes easy IR-tree junk early so the same downstream optimizations have more room to apply and fewer useless instructions to revisit.

5.2 Repeated Scalar Cleanup After Major Transformations

A recurring design choice in our optimizer is to run `basic::optimize` after larger transformations. This cleanup pass performs propagation, branch folding, strength reduction, label coalescing, and DCE. The reason is that larger passes often expose simpler code patterns that are not visible before the transformation.

To test this interaction, we compared the full `-O1` pipeline against an ablation that keeps only the first scalar cleanup pass and skips the intermediate cleanup calls after major transformations. Positive Δ cycles means the full repeated-cleanup pipeline was faster; positive static instruction delta means the ablated compiler emitted more static instructions.

Benchmark	Cycles no cleanup	Cycles full -01	Δ cycles	Static instr. Δ	Dominant interaction
julia.14	1,850,162,854	1,748,485,135	101,677,719	49	IV strength + simplify
jack.14	105,902,703	56,363,213	49,539,490	78	inline + DCE/prop
daisy.14	211,921,187	186,133,020	25,788,167	234	loop opts + fold
georgy.14	139,455,031	117,815,906	21,639,125	37	inline + DCE/prop
ncik.14	436,236,703	433,080,500	3,156,203	67	IV strength + simplify
fannkuch.14	2,069,637,203	2,103,742,755	-34,105,552	4	backend interaction/noise

Table 11: Effect of repeated scalar cleanup after major transformations.

The largest gain is on `julia.14`. This benchmark contains several arithmetic-heavy loops: Julia-set rendering repeatedly computes scaled coordinates, complex multiplication, divisions by constants, and Collatz-style updates such as `n % 2`, `n / 2`, and `3*n+1`. These are exactly the kinds of expressions where strength reduction, IV-style simplification, and DCE interact. Without intermediate cleanup, many of the simplification opportunities exposed by those rewrites remain in the final code, leaving 49 extra static instructions and costing over 10^8 cycles.

`jack.14` shows the inlining-cleanup interaction. Its max-flow implementation repeatedly calls small helpers such as `min`, `enqueue`, `dequeue`, and `is_square` inside BFS, edge-construction, and flow-update loops. After inlining these helpers, propagation and DCE can remove argument copies, simple return-value moves, and dead branch structure. Skipping intermediate cleanup leaves 78 extra static instructions and nearly doubles the measured runtime for this benchmark.

5.3 LICM and Induction-Variable Strength Reduction

To test a true interaction, we measured LICM and induction-variable strength reduction separately and together. LICM moves loop-invariant work into preheaders, while IV strength reduction replaces repeated induction-variable multiplication with a loop-carried value updated by addition. We ran four configurations: both disabled, LICM only, IV only, and both enabled, keeping the rest of the optimizer fixed. Synergy is computed as

$$\text{synergy} = \text{improvement}(\text{LICM}+\text{IV}) - \text{improvement}(\text{LICM}) - \text{improvement}(\text{IV}),$$

where each improvement is relative to the both-off baseline.

Table 12: Synergy between LICM and induction-variable strength reduction.

Benchmark	Both off	LICM only	IV only	LICM+IV	Synergy
ncik.14	432,203,771	428,783,208	832,623,937	428,198,625	401,004,749
arrays_and_loops.14	942,234,906	1,126,273,979	1,029,323,260	1,002,677,937	210,684,396
julia.14	1,661,624,119	1,726,505,172	1,748,995,260	1,624,641,260	189,235,053
loooops.14	2,043,475,833	2,054,894,354	2,203,730,416	2,087,649,995	127,498,942
mat.14	242,952,526	124,093,172	243,820,765	122,875,135	2,086,276
daisy.14	203,313,166	185,169,635	215,676,088	301,336,578	-103,804,021

The clearest useful case is `julia.14`: LICM alone and IV alone both regress, but together they beat the both-off baseline by about 3.7×10^7 cycles. This fits the source structure, which has nested rendering loops, scaled-coordinate arithmetic, and Collatz-style updates. `mat.14` is different: LICM alone captures almost all the benefit, so the pair has little extra synergy.

The large positive synergy values for `ncik.14`, `arrays_and_loops.14`, and `loooops.14` should be read carefully: IV alone is very harmful there, so the synergy number partly means that enabling LICM avoids the worst IV-only behavior, not that the pair always gives a large absolute win. `daisy.14` shows the opposite interaction: LICM helps alone, but combining it with IV strength hurts badly, likely because its mesh-style pointer traversal and irregular loop bounds are a poor match for the IV rewrite. LICM and IV strength are clearly not independent; their usefulness depends heavily on loop shape and pass ordering.

6 Compile Time Optimizations

6.1 String Interning

Identifier names are interned on parsing: each unique string is stored once in a hash table, and all subsequent references use a pointer comparison rather than a full string comparison. This eliminates redundant string hashing and copying throughout the pipeline — in symbol table lookups, variable resolution, and label matching. The effect is most pronounced on benchmarks with many identifiers or deeply nested scopes, where symbol table queries dominate compile time.

6.2 Dense dominator bitvectors

We represent each block’s dominator set as a fixed-length vector of 64-bit words, with one bit per basic block, instead of storing a cloned ordered set of predecessors for every node. This avoids the textbook initialization that duplicates a full “all blocks” set at every CFG node, which costs quadratic memory and heavy allocator traffic on large flattened functions. Dominance queries and updates therefore scan or intersect compact bitmasks, which reduces both the time and the peak memory of passes that recompute dominators on big LLVM IR bodies.

6.3 Single-Benchmark Ablation Summary

Finally, Figure 2 summarizes the main ablation results for `julia.14`. Each bar reports

$$\Delta\text{cycles} = \text{cycles with optimization disabled} - \text{cycles with full -O1}.$$

Thus, positive bars mean the optimization helped, because disabling it made the benchmark slower; negative bars mean the optimization hurt on this benchmark.

This figure is useful because `julia.14` stresses many parts of the optimizer at once: arithmetic strength reduction, inlining, repeated scalar cleanup, peephole cleanup, and loop-heavy numeric code.

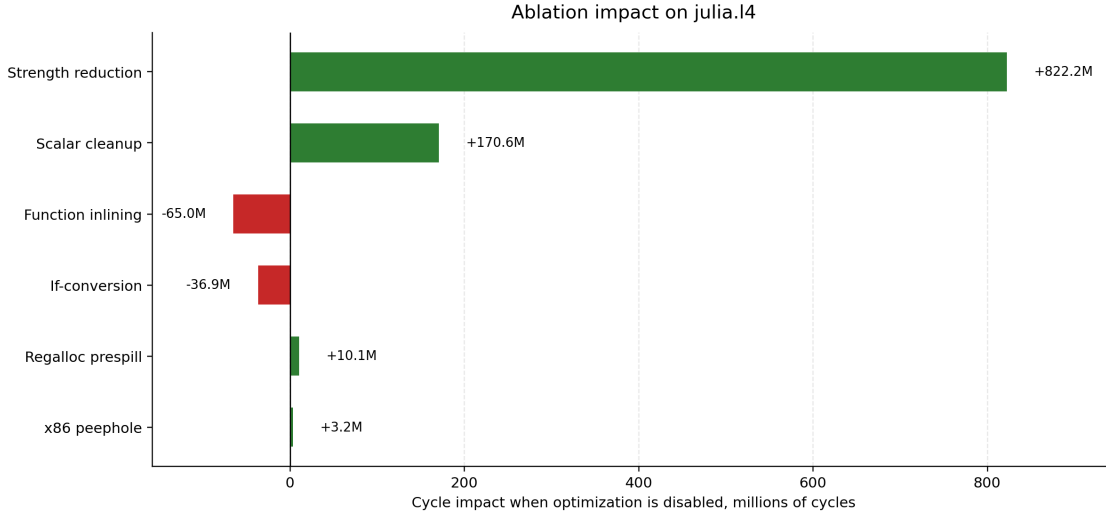


Figure 2: Ablation impact on julia.l4

7 Limitations and future work

Several passes are conservative by design. PRE (`llvm_optim/pre.rs`) omits ϕ -insertion (diamond joins skipped) and caps candidates/body size. `pass_load_cse` (`llvm_optim/basic.rs`) clears all cached loads on any `Store` or `Call`. Scalar replacement (`llvm_optim/scalar_replacement.rs`) only follows simple derived-pointer chains (`safe_use`). LICM (`llvm_optim/licm_pass.rs`) tapers rounds on huge functions; `llvm_optim/mod.rs` reruns LICM and `basic::optimize` around other phases. IV strength (`llvm_optim/iv_strength.rs`), inlining (`llvm_optim/inline.rs`), and TCO (`llvm_optim/tco.rs`) cover narrow patterns only. `llvm_optim/basic.rs` skips folding trapping divides and falls back to DCE-only on very large bodies; `ir_opt.rs` leaves dead-block removal to LLVM and defers shifts. Future work: alias / value-range reasoning, fuller PRE and IV/TCO, richer inlining, and stronger regalloc coalescing (`regalloc.rs`). Value Range Analysis is also not implemented in any capacity.

References

- [1] Torbjörn Granlund and Peter L. Montgomery. Division by invariant integers using multiplication. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation (PLDI '94)*, pages 61–72, 1994. <https://doi.org/10.1145/178243.178249>
- [2] Henry S. Warren, Jr. *Hacker's Delight*. 2nd edition, Addison-Wesley Professional, 2012.
- [3] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. 2nd edition, Morgan Kaufmann, 2011.
- [4] Fernando Magno Quintão Pereira and Jens Palsberg. Register allocation via coloring of chordal graphs. In *Proceedings of the Asian Symposium on Programming Languages and Systems (APLAS)*, pages 315–329, 2005. https://doi.org/10.1007/11575467_21
- [5] LLVM Project. Tail recursion elimination pass. https://llvm.org/doxygen/TailRecursionElimination_8cpp.html
- [6] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on*

Programming Languages and Systems, 13(4):451–490, 1991. <https://doi.org/10.1145/115372.115320>